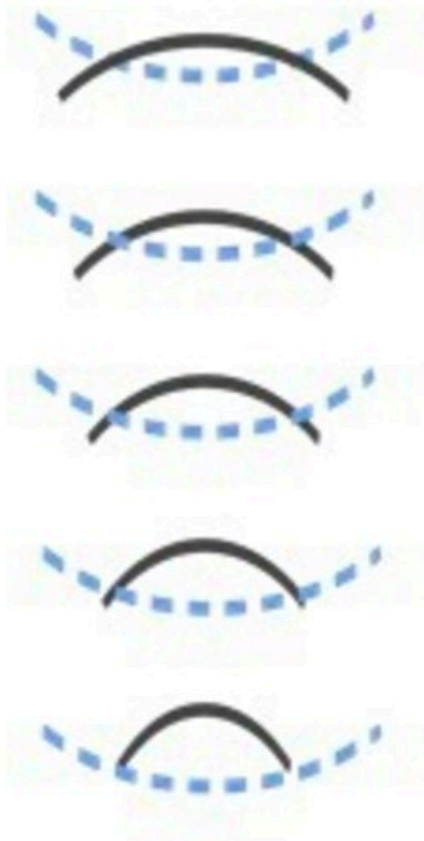


Task 1: Binary Classification





Task 1 is to implement a perceptron classifier that can handle a binary classification task (two class labels). The task that your perceptron will learn is: given some sonar data, determine whether the object detected by the sonar is a rock or an underwater mine.

Set Up

1. Download the code from [\[here\]](#) by downloading the whole Classification folder and then extracting it into your Documents folder.
2. Open the code in VS Code by doing File > Open Folder. Select the top-level **Classification** folder.
3. Once it is open in VS Code, you will see a folder called **Exercises**. This is where your code will go. Do not modify any code outside of the Exercises folder!
4. Open the file called `perceptron_binary.py` in the **Exercises** folder. You'll see that there are two functions you will need to implement in order to create your binary perceptron classifier.

Training Data

Each record contains:

- 60 numbers in the range 0.0 to 1.0. Each of these features represent the energy within a particular frequency band.
- A label with 'R' for rock or 'M' for mine. When we read in the dataset, we convert '**R**' to **0** and '**M**' to **1** (this is done for you). The label is in the last column of the data.

For example, a training record will look like this:

```
0.1021, 0.083, 0.0577, 0.0627, 0.0635, ..., 0.0177, 0.0214, 0.0227,  
0.0106, 1
```

You will use this labeled data to train your perceptron to learn the difference between rocks and mines. Let's get started!

Functions to implement

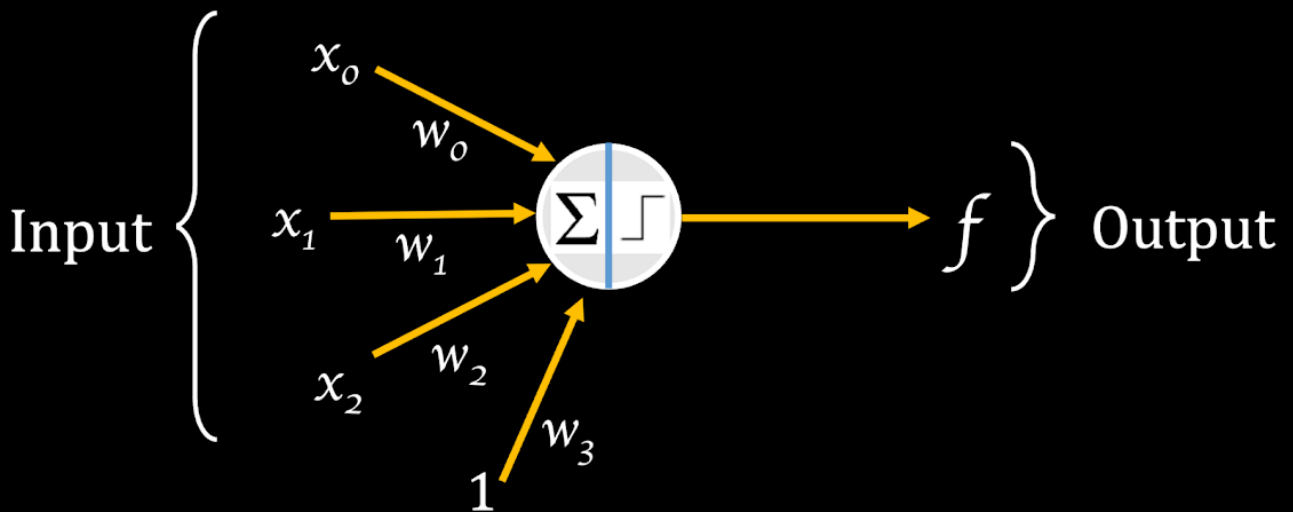
1. `classify()`



The first function, called `classify()`, takes in as parameters, a row, which is a list of feature values for one record, and a list of weights, which are the current weight values for the perceptron.

Use those inputs to return a predicted class label for the row that is passed in to this function. Class 0 represents 'Rock' and class 1 represents 'Mine'.

Remember how a perceptron predicts a class label...



$$f = \begin{cases} 0 & \text{if } x_0w_0 + x_1w_1 + x_2w_2 + w_3 < 0 \\ 1 & \text{if } x_0w_0 + x_1w_1 + x_2w_2 + w_3 \geq 0 \end{cases}$$

2. `train()`

The second function, called `train()`, is where the perceptron will adjust its weights as it learns from the training data. This function takes in the labeled training data set, called `train_data`, and a number of epochs to train for, called `n_epochs`.

With this, you should implement the code to update and learn the perceptron's weights, based on the training data.

For the initial weights, go ahead and set them as random values between -1 and 1. This can be done by using `uniform(-1, 1)`

Remember the steps of the perceptron learning algorithm we discussed in lecture.

If you need a hint or need help with this part, flip your cups to red!

This function should return a list of the final weights, once the perceptron has trained on all training records for `n_epochs`.

Also, I want you to add into this function a printout that shows the epoch number and what percentage of the records were classified correctly during this epoch. So, when this code is run, you should see printouts that look like this - this is how we can watch it learn!

```
epoch 0 .... 51% correct.  
epoch 1 .... 66% correct.  
epoch 2 .... 71% correct.  
epoch 3 .... 73% correct.  
...
```

To run your code, use the command:

```
python3 run_perceptron_binary.py
```

This will run a python file that runs your `perceptron_binary.py` code.

Once you have completed this task, turn your cups to red so that a member of the staff can check your code.

